Newton School of Technology

2

Classes and Objects

By - Aditya Kumar

Newton School of Technology

Join the lecture online on your dashboard.

Have you played this game?

If we want to create our own BGMI?

Newton School of Technology

Can you think of few things we will need to create, in order to have a fully functional BGMI? ……

If we want to create our own BGMI?

Let’s Start with designing the below given Items

What all properties the player will have?

What all properties the player will have?

- Name
- Health
- Current weapon
- Current Vehicle
- isAlive

What all things a player can do?

What all things a player can do?

- Pick Weapon
- Drop Weapon
- Enter Vehicle
- Exit Vehicle
- Fire

These are called **Behaviour - Let's call it this is what a player can do or does…**

What properties a car will have?

What properties a car will have?

- Model
- Color
- Number of Wheels
- Seating Capacity
- And few more….

What functionalities a car will have?

What functionalities a car will have?

- Start
- Stop
- Accelerate
- Brake
- And few more….

How will we code our Car?

Newton School of Technology

<CODE/>

How will we code our Car?

```
let car = {
    model: "Jeep",
    color: "Gray",
    wheels: 4,
    seatingCapacity: 5,
    accelerate: function() {
        console.log("The car is accelerating.");
    },
    break: function() {
        console.log("The car is braking.");
    }
}
```

How many cars you will create?

May be 100?

How many cars you will create?

May be 100?

Should we create 100 objects one by one?

How many cars you will create?

May be 100?

Or we have a better option?

How many cars you will create?

May be 100?

Can we create a blueprint for this?

Blueprint/Template

Blueprint

Classes & Objects

Newton School of Technology

Class = Blueprint / Template

A **class** is a *blueprint* or *template* that defines:

- **Attributes** (data / variables)
- **Behaviours** (functions / methods)

Blueprint/Template

Blueprint using Class

```
class Car {
  model: string;
  color: string;
  wheels: number;
  seatingCapacity: number;

  constructor(model: string, color: string, wheels: number, seatingCapacity: number) {
    this.model = model;
    this.color = color;
    this.wheels = wheels;
    this.seatingCapacity = seatingCapacity;
  }

  accelerate() {
    console.log("The car is accelerating.");
  }

  brake() {
    console.log("The car is braking.");
  }
}
```

How will we use this Blueprint?

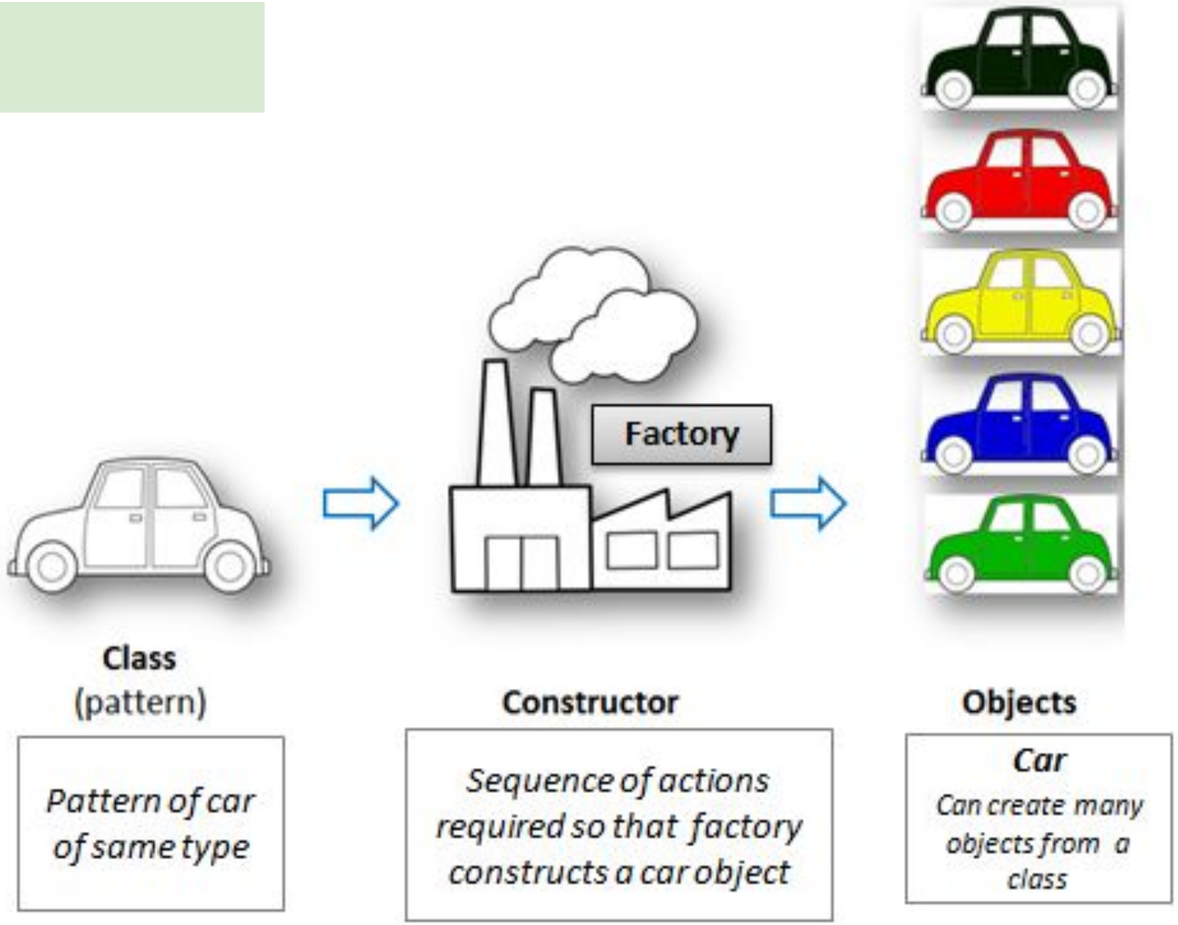
```
const car = new Car("Jeep", "Gray", 4, 5);
car.accelerate();
car.brake();
```

Constructors

Constructor = Special Function

- Runs **automatically** when an object is created

- Sets initial **state** of the object
  Ensures the object is in a **valid** and usable condition



What will happen if we don’t use constructor?

If we didn’t write constructor

Who did ?

You can write a constructor for your class, but if you don’t, the compiler writes one for you!

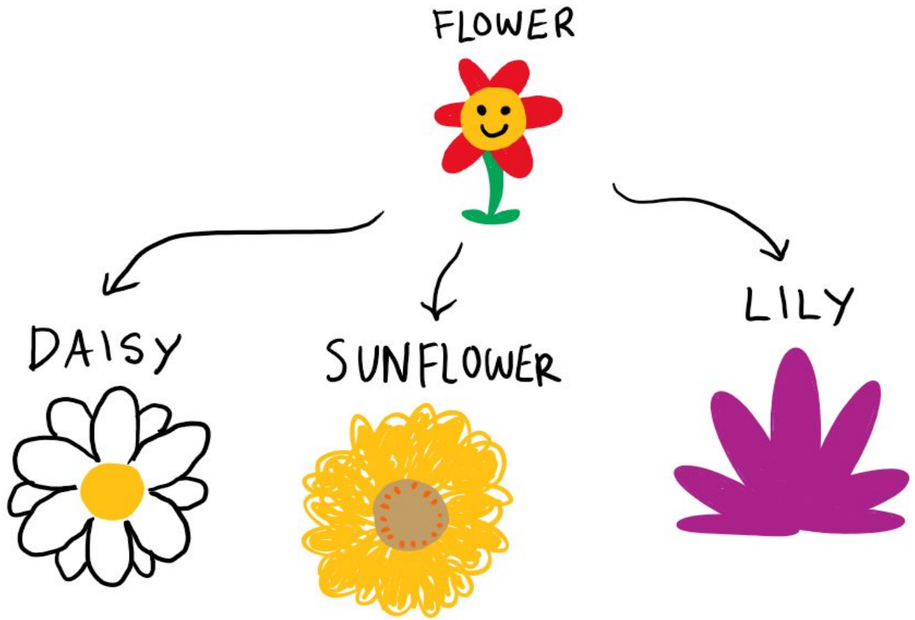
Here’s what the compiler’s **default constructor** looks like:

```
constructor() {}
```

Classes & Objects

Object = Actual built item based on the blueprint

- A **real instance** of a class
- Something that **occupies memory**
- Something you can **interact with**
- Has its own **state** and **behaviour**



A class is only a plan — the **object is the actual item** created from that plan.

Do you want your car to have Zero seating capacity?

Invariants

Newton School of Technology

Your car should have at least 1 seat.

Invariants

How can we make sure the seating capacity should be always greater than ZERO.

Invariants

```
constructor(model: string, color: string, wheels: number, seatingCapac
  if (seatingCapacity <= 0) {
    throw new Error("seatingCapacity must be > 0");
  }
```

Why are Invariants important ?

- Consistent and Predictable Object Behavior

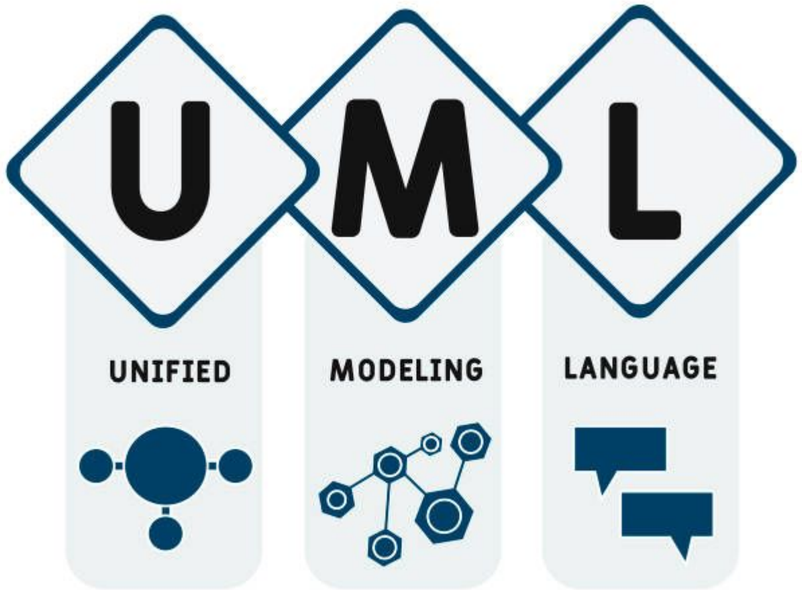
- Fewer Bugs Through Early Error Detection

- Better Maintainability with Clear Rules

Which one occupies memory: Class or Object?

UML

- UML = a **visual language** for modeling software systems.
- **Technology-agnostic**: works for Java, TypeScript, Python, C#, etc.



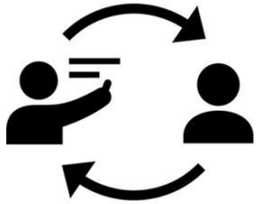
Why UML Matters ?

- **“A picture of the system”** → easier than reading thousands of lines of code.

- Great for planning before writing code.

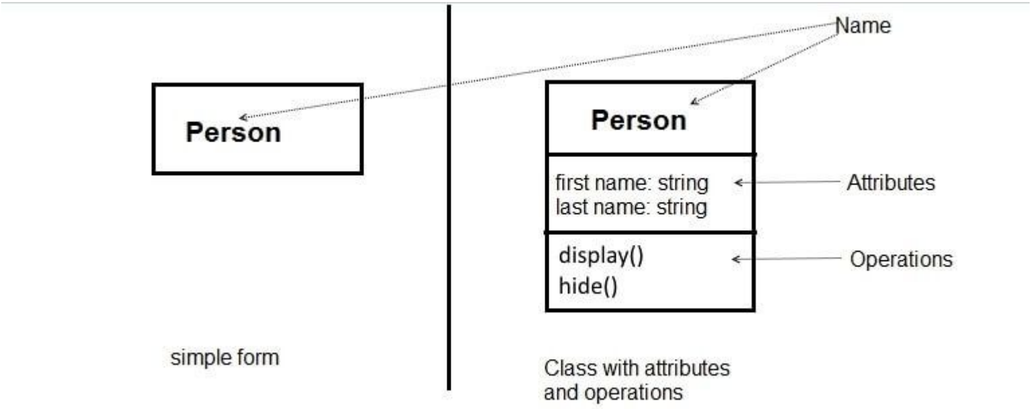
Why use UML ?

- Clear Communication
- Design Blueprint
- Manage Complexity
- Industry Standard



UML class diagram example

Library
books: string[] location: string
constructor(location: string) addBook(book: string): void removeBook(book: string): void getBooks(): string[] getLocation(): string



Student
name: String age: number
constructor(name: string, age: number) setName(name: string): void setAge(age: number): void getAge(): number getName(): string

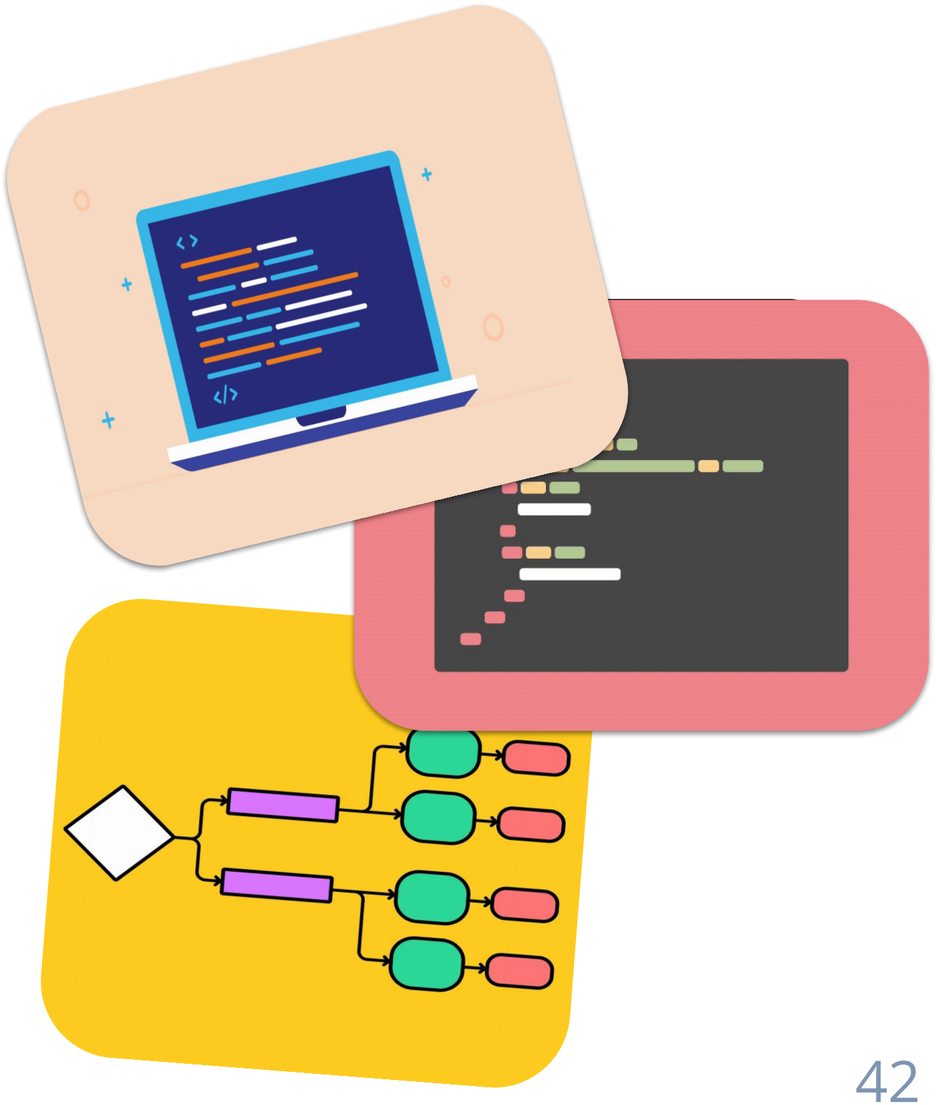
MCQ Time ..

Summary

Everything in OOP is modelled as **objects** with **state, behaviour, identity**.

Constructors create objects and ensure invariants.

UML helps visualize and plan class structure.



THANK
YOU!